HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

INTERNATIONAL MASTER PROGRAM

COURSE: PARALLEL COMPUTING

ASSIGNMENT 1

Lecturer: **Dr. Diep Thanh Dang.**

Author: Nguyen Dinh Phu - 2470891

HO CHI MINH CITY, 2026

Table of contents

1. Mutex Lock **3**
1.1. Implement TAS/ TTAS/ MCS using C++ 3
1.2. Test the correctness 3
1.3. Test mutex locks with is_prime code fragment 3
● Analyze TAS result 4
● Analyze TTAS result 5
● Analyze MCS result 7
● Impact of exponential backoff on execution time 8
1.4. Performance comparison with Pthreads & OpenMP 10
● TAS with Pthreads and OpenMP 10
● TTAS with Pthreads and OpenMP 12
● MCS with Pthreads and OpenMP 13
2. Barrier Synchronization **14**
2.1. Implement Sense-Reversing Barrier (SRB) 14
2.2. Test the correctness 14
2.3. Performance and Scalability Analysis 14
3. Concurrent Counters **16**
3.1. Implement concurrent counters using mutex locks, compare-and-swap, fetch-and-add. 16
3.2. Test the correctness 16
3.3. Performance and Scalability Analysis 17
4. References **20**

1. Mutex Lock

1.1. Implement TAS/ TTAS/ MCS using C++

See the attachment (under *mutex-lock* folder)

1.2. Test the correctness

A simple and effective way to test the correctness of our mutex lock implementations (TAS, TTAS, MCS) is to use a shared counter benchmark. I initialize a global integer counter to zero and start T worker threads; each thread executes a loop of N iterations where it acquires the lock, increments the shared counter once inside the critical section, and then releases the lock.

At the end of the run, the main thread joins all workers and checks whether the final counter value equals the mathematically expected result T×N. If the lock fails to provide mutual exclusion, some increments will be lost (two threads entering the critical section at the same time), and the final value will be smaller than T×N, which directly reveals race conditions or memory-ordering bugs in the implementation.

By repeating this test for different thread counts and contention levels (varying outside work), I increase the chance of exposing subtle errors while still having a very clear, deterministic correctness criterion: the shared counter must always equal T×N for all lock variants.

The test code is presented in the attachment (*test_mutex_lock.cpp*)

1.3. Test mutex locks with is_prime code fragment

The experiments were run on a shared-memory machine with up to 21 hardware threads. For each configuration, I used 1,000 iterations per thread and varied the number of threads (1, 2, 4, 8, 12, 16, 20), the contention level via outside work (0, 1, 3, 5, 7, 9), and the critical-section cost via the prime input n in (53, 89, 101, 503, 1009, 10007, 50021, 100003). To reduce measurement noise, every configuration was executed 20 times and I report the mean total execution time over these 20 runs. After collecting all measurements, I classified for each configuration which lock (TAS, TTAS, or MCS) achieved the lowest execution time and then analyzed these winning cases to understand under which conditions each lock type tends to perform best.

- Analyze TAS result

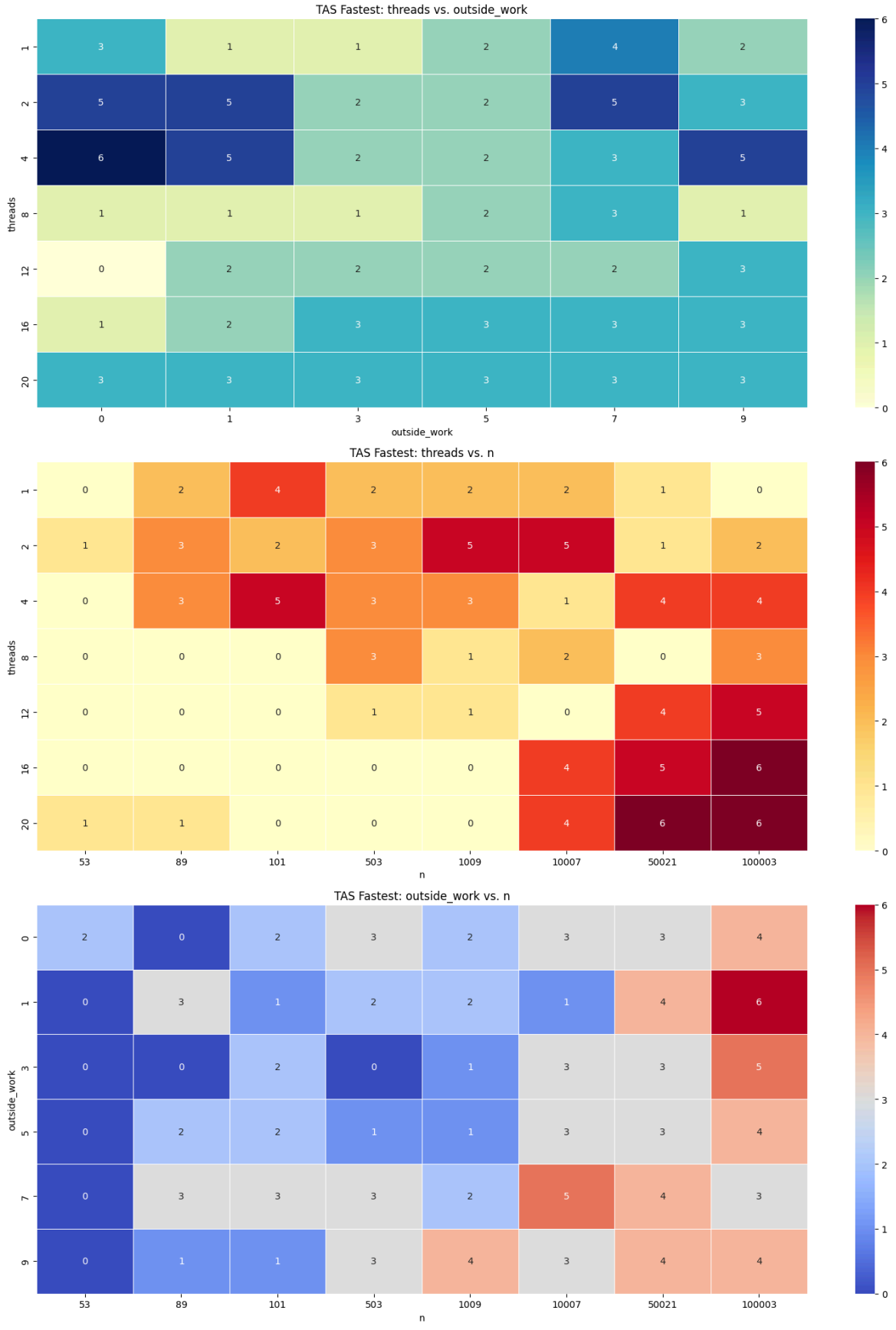


Figure 1: The heatmap number of cases TAS is fastest (compare on different views).

As shown in figure 1, TAS shows a clearer advantage as the critical section becomes larger, which in this experiment is reflected by larger values of n. Across the heatmaps, TAS wins are concentrated much more strongly at medium-to-large n, especially 10007, 50021, and 100003, while it rarely wins at very small n such as 53. This suggests TAS benefits when the useful work inside the critical section dominates the locking overhead.

The effect of thread count is less uniform than the effect of n. TAS performs reasonably well at low-to-moderate thread counts such as 2 and 4, but it also remains competitive at 16 and 20 threads when n is large. TAS does not depend on one narrow outside_work setting. its wins are spread across different outside_work values, with a slight concentration at higher outside_work such as 7 and 9. This indicates TAS is not tied to a single contention pattern, but instead benefits more from workloads where lock overhead becomes relatively less important.

Overall, the TAS heatmaps suggest that TAS is most suitable when the **critical section is long** and the lock cost is amortized by larger computation.

- Analyze TTAS result

The heatmap 1 (threads vs outside work) and heatmap 3 (outside work vs n) in the figure 2 show a consistent pattern for TTAS. In both views, TTAS performs best when outside_work is very small, especially at outside_work = 0, which corresponds to frequent lock re-attempts and high contention. Heatmap 1 shows that under this condition, TTAS can win even at higher thread counts such as 8, 12, 16, and 20, while Heatmap 3 confirms that this advantage is tied more strongly to low outside_work than to any single n range. Together, these two heatmaps suggest that TTAS benefits most when threads repeatedly compete for the lock with little delay between attempts.

Heatmap 2 (threads vs n) looks different because it aggregates over all outside_work values and therefore hides the narrow condition that makes TTAS strong. Although TTAS can perform well at high thread counts when outside_work = 0, that advantage does not hold across larger outside_work values. As a result, once all outside_work settings are combined, high-thread TTAS wins become less frequent overall, and the heatmap shows more wins at lower thread counts such as 2 and 4. This is an aggregation effect: a strong pattern in one specific slice of the data becomes diluted when another dimension is collapsed.

Overall, TTAS advantage is in **high-contention settings** where outside_work is minimal, but that advantage weakens when contention is reduced by additional outside work.

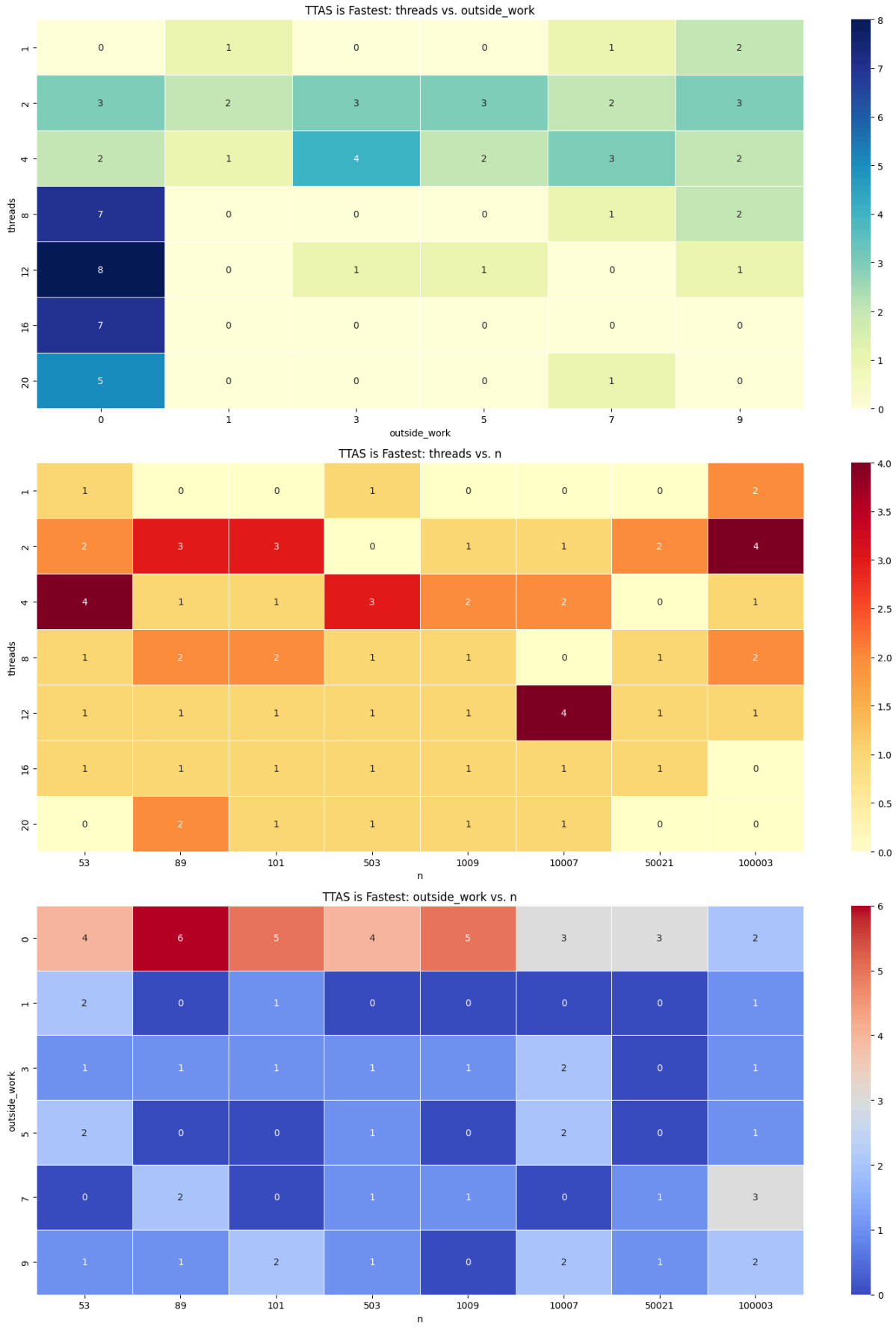


Figure 2: The heatmaps for the number of cases TTAS is fastest.

- Analyze MCS result

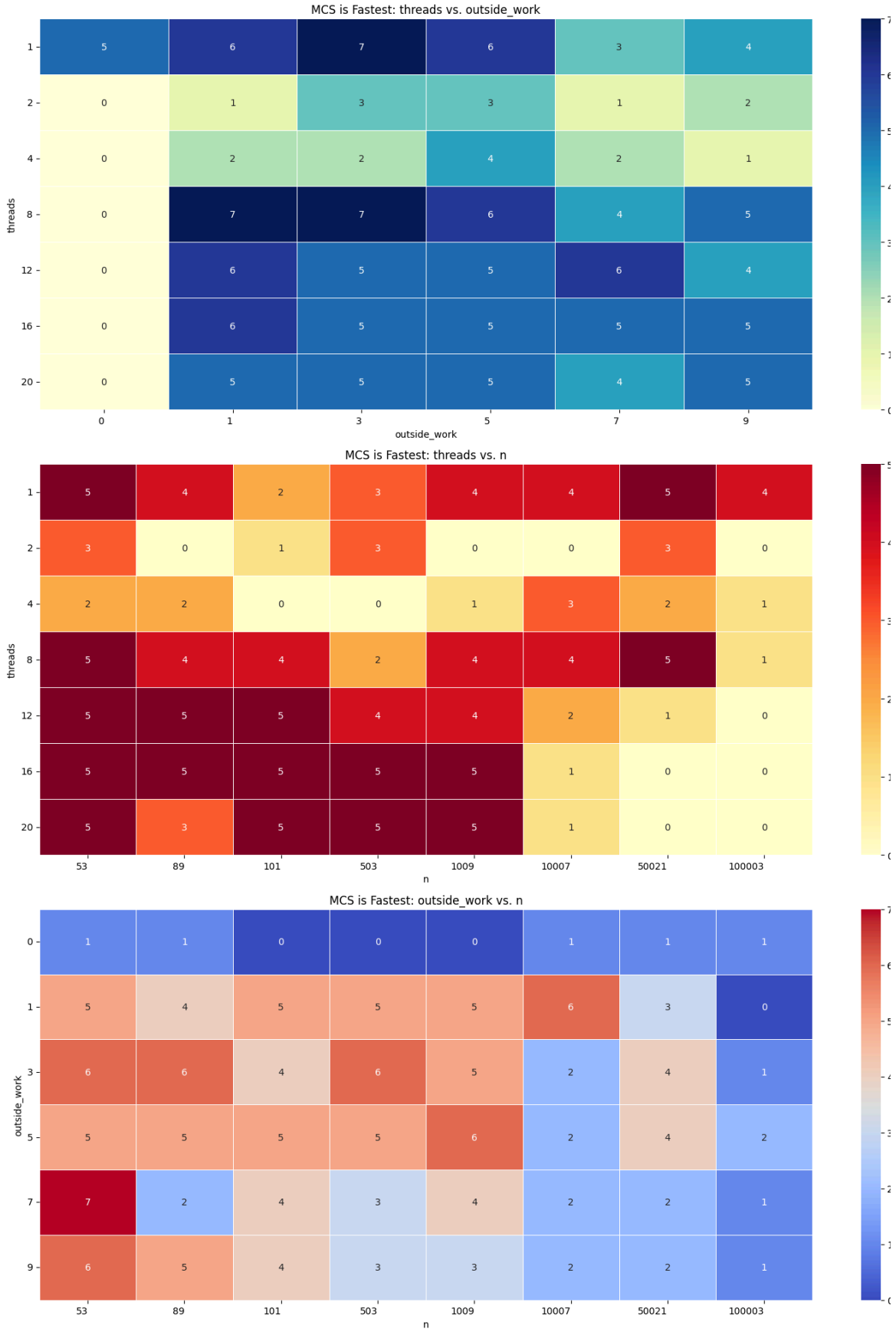


Figure 3: The heatmaps for the number of cases MCS is fastest.

As shown in figure 3, MCS performs best mainly when the critical section is short to moderate, which is reflected by smaller values of n. Across the heatmaps, MCS wins are concentrated at low n such as 53, 89, 101, 503, and 1009, while its advantage drops noticeably as n becomes very large, especially at 100003. This suggests MCS is effective when lock-management efficiency matters more than the amount of work inside the critical section.

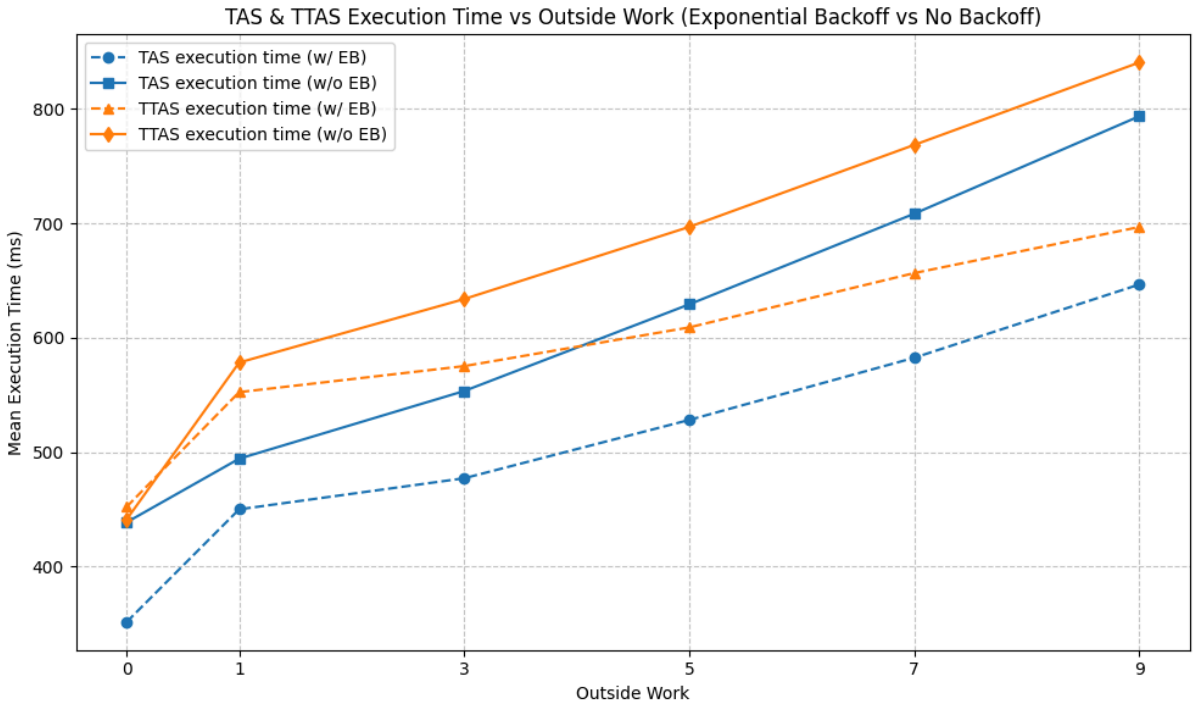
In terms of contention, MCS shows a fairly stable advantage across medium to high thread counts, especially 8, 12, 16, and 20 threads. Unlike TTAS, MCS does not benefit from outside_work = 0; in fact, it wins much more often when outside_work is moderate, particularly at 1, 3, and 5. This indicates that MCS is well suited to settings with sustained contention and more balanced workloads, where fairness and orderly lock handoff help reduce the cost of contention.

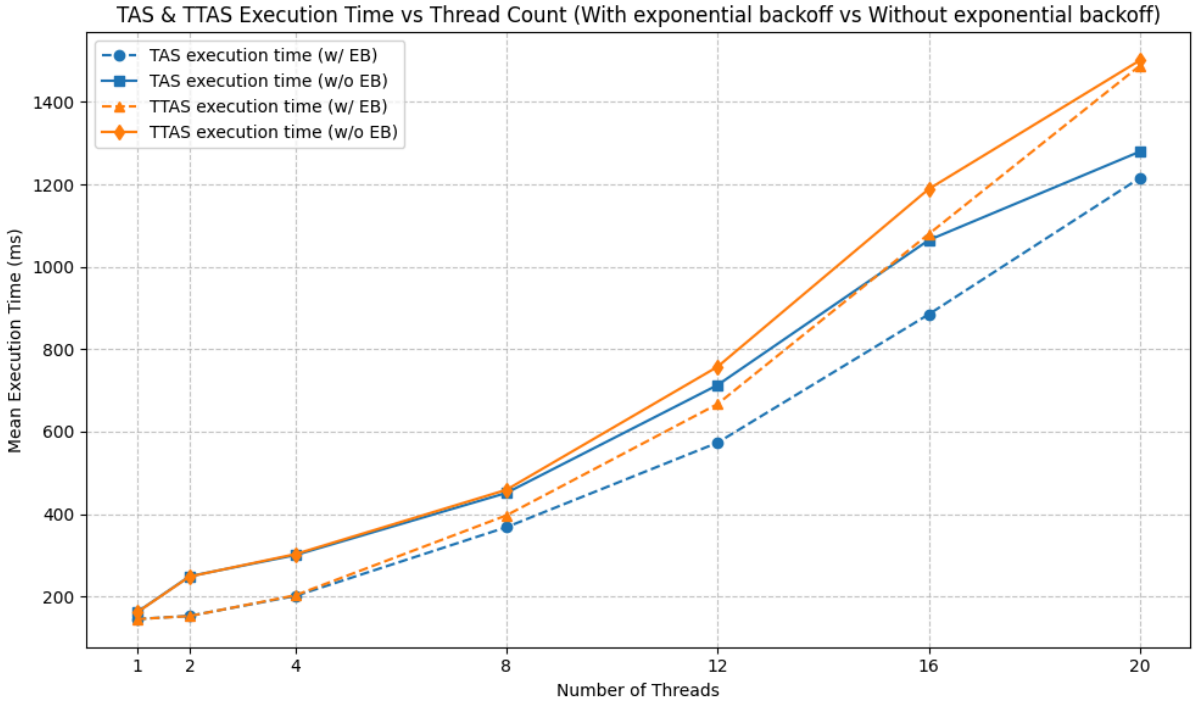
MCS wins in approximately 46% of all cases, highlighting its robustness across a wide range of scenarios. The MCS heatmaps support the conclusion that MCS is the most robust lock for **high-contention situations with relatively short to moderate critical sections**. Its advantage is mainly due to its scalable behavior under many competing threads, rather than performance in locks with long critical sections. This makes MCS a strong choice for scenarios where contention control and fairness are more important than lock simplicity or overhead.

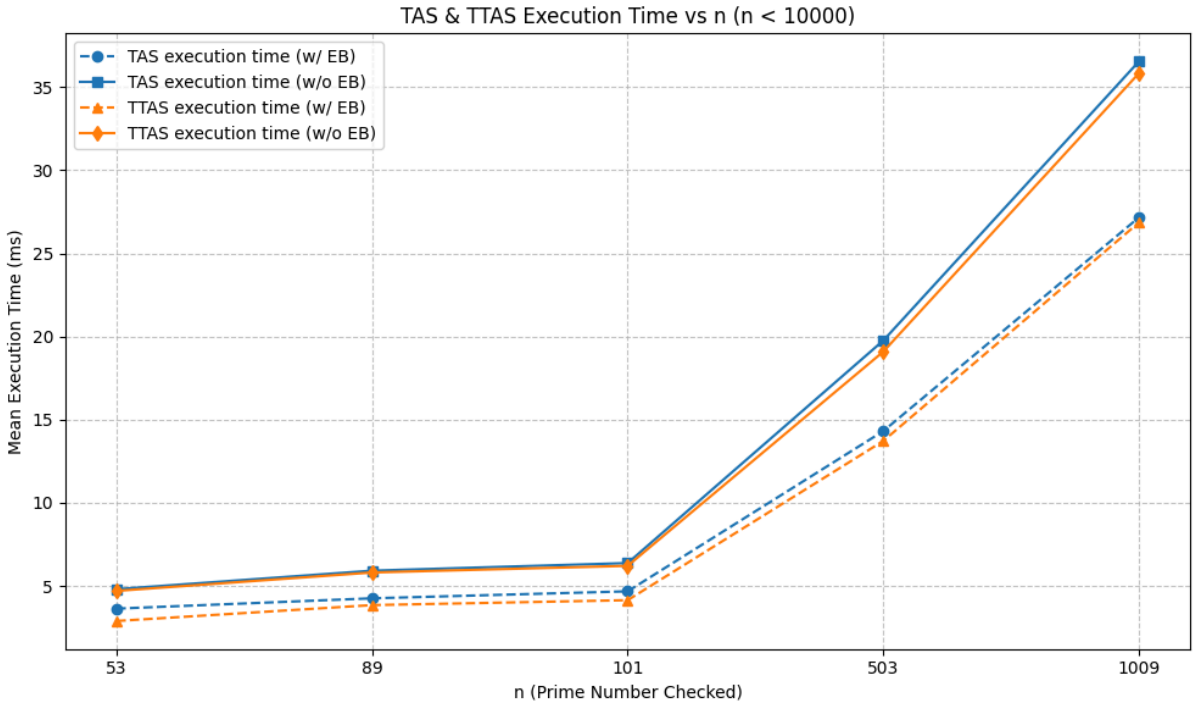
- Impact of exponential backoff on execution time

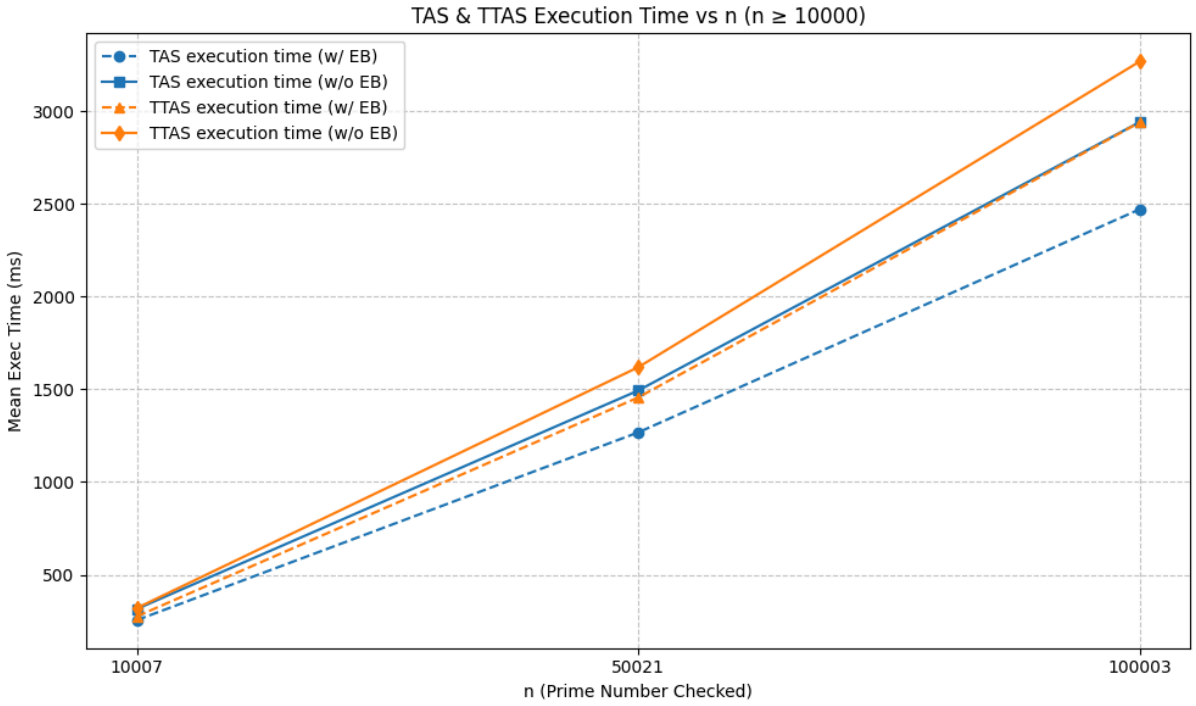
Empirical results from the experiments demonstrate that enabling exponential backoff significantly reduces the execution time of both TAS and TTAS locks, independent of the number of threads, the amount of outside work, or the size of the prime number checked (n).

Across all tested configurations - ranging from single-threaded to high-concurrency environments, for both small and large values of n - the mean execution time is consistently lower when exponential backoff is used. This effect is visible in the plots presented below and can be attributed to the backoff mechanism reducing contention and unnecessary spinning, leading to improved resource usage and lower lock acquisition latency.


TAS & TTAS Execution Time vs Outside Work (Exponential Backoff vs No Backoff)
TAS execution time (w/ EB)
TAS execution time (w/o EB)
TTAS execution time (w/ EB)
TTAS execution time (w/o EB)
Mean Execution Time (ms)
400
500
600
700
800
0
1
3
5
7
9
Outside Work


TAS & TTAS Execution Time vs Thread Count (With exponential backoff vs Without exponential backoff)
TAS execution time (w/ EB)
TAS execution time (w/o EB)
TTAS execution time (w/ EB)
TTAS execution time (w/o EB)
Mean Execution Time (ms)
200
400
600
800
1000
1200
1400
1
2
4
8
12
16
20
Number of Threads





1.4. Performance comparison with Pthreads & OpenMP

- TAS with Pthreads and OpenMP

To compare our custom TAS spinlock with standard library mechanisms, I evaluated lock variants on the same benchmark. I focused on a low-contention, heavy critical section regime where the critical section dominates runtime: number of iteration is

1000, thread counts is in predefined range (2, 4, 12, 16), outside work is 7 and 9, and large prime inputs (10007, 50021, 100003). For each configuration, I measured the total execution time of the whole program in milliseconds.

Across the 24 configurations in this regime, TAS achieved the lowest execution time in 8 cases, while in the remaining 16 cases either the Pthreads or OpenMP mutex was marginally faster or essentially tied with TAS.

The fact that TAS is fastest in only one third of the cases, and by a small margin, also shows that its advantage in this regime is situational rather than systematic. Since the workload is dominated by computational cost rather than lock contention, all four mechanisms exhibit similar overall performance, and modest constant-factor differences in their implementations are enough to change which lock is slightly better for a particular parameter combination. I, therefore, conclude that, under low contention with long critical sections, TAS spinlock is competitive with Pthreads and OpenMP locks but not clearly superior, standard mutexes remain a reasonable choice.

Table 1: The comparison between TAS vs. Pthreads/OpenMP
(highlighted rows indicate winning cases)

threads	num_iters	outside_work	n	tas_ms	tas_backoff_ms	pthread_mutex_ms	omp_lock_ms
2	1000	7	10007	109.848	109.818	107.004	108.67
2	1000	7	50021	588.968	564.917	551.443	543.552
2	1000	7	100003	1128.48	1160.15	1131.68	1134.13
2	1000	9	10007	154.238	136.671	135.936	135.262
2	1000	9	50021	727.216	688.76	706.774	693.38
2	1000	9	100003	1466.04	1424.2	1458.95	1438.37
4	1000	7	10007	131.638	133.508	118.798	119.248
4	1000	7	50021	693.735	656.044	614.558	603.99
4	1000	7	100003	1310.02	1382.97	1401.4	1416.27
4	1000	9	10007	193.803	198.296	175.185	174.682
4	1000	9	50021	958.48	986.01	993.898	1002.94
4	1000	9	100003	2031.98	2000.11	1775.47	2704.42
12	1000	7	10007	575.082	370.728	381.739	476.938
12	1000	7	50021	1888.88	1838.9	1426.9	1437.11
12	1000	7	100003	3475.57	3602.05	2776.36	2730.45
12	1000	9	10007	367.671	389.437	369.727	384.17
12	1000	9	50021	1885.11	1850.29	1545.33	1568.37
12	1000	9	100003	3784.16	3821.87	2989.05	3061.51
16	1000	7	10007	504.703	521.128	446.437	464.813
16	1000	7	50021	2419.06	2483.11	1876.21	1945.57
16	1000	7	100003	4804.43	4949.11	3655.29	3695.58
16	1000	9	10007	488.606	539.538	509.321	504.239
16	1000	9	50021	2700.74	2682.54	2056.73	2046.8
16	1000	9	100003	5153.3	5310.12	4014.37	4042.07

- TTAS with Pthreads and OpenMP

For TTAS, I focused on the regime where it should work best: many threads, maximum contention, and short critical sections: number of iterations is 1000, outside work is 0, threads count is in range (8, 12, 16, 20) and small primes n.

TTAS consistently achieved the lowest execution time (see Table 2), often significantly faster than both Pthreads and OpenMP. For example, at 20 threads and n = 503, TTAS completed in about 20 ms, while Pthreads needed roughly 37 ms and the OpenMP lock more than 120 ms. This matches the theoretical expectation that TTAS, which mostly spins on read-only loads and performs fewer failed test-and-set operations, is well suited for highly contended, very short critical sections.

Table 2: The comparison between TTAS vs. Pthreads/OpenMP
(highlighted rows indicate winning cases)

threads	num_iters	outside_work	n	ttas_ms	ttas_backoff_ms	pthread_mutex_ms	omp_lock_ms
8	1000	0	53	1.8001	1.3847	1.7198	42.4612
8	1000	0	89	1.6907	1.7361	2.2347	40.0963
8	1000	0	101	2.2664	1.89	2.9447	40.6828
8	1000	0	503	6.8324	6.993	8.3542	45.0527
8	1000	0	1009	12.6582	12.409	15.4098	50.9043
12	1000	0	53	3.9416	2.3595	2.8351	61.5107
12	1000	0	89	3.5445	3.1508	4.4061	62.8281
12	1000	0	101	4.3539	2.4348	4.3945	64.5522
12	1000	0	503	11.2629	10.0983	13.7846	73.3708
12	1000	0	1009	21.6419	21.6769	21.4901	88.7702
16	1000	0	53	3.4991	3.4562	3.649	89.3634
16	1000	0	89	3.4711	2.9475	5.5038	82.3
16	1000	0	101	7.5956	4.5901	6.3911	83.2098
16	1000	0	503	15.3839	13.4796	16.7352	90.4785
16	1000	0	1009	28.1073	28.9907	29.2095	101.355
20	1000	0	53	7.4591	3.3645	4.7973	105.212
20	1000	0	89	6.8095	4.6821	7.6315	104.243
20	1000	0	101	7.7479	6.4256	7.5973	106.223
20	1000	0	503	20.4089	17.3956	21.3001	116.547
20	1000	0	1009	36.0365	36.988	36.6165	126.936

- MCS with Pthreads and OpenMP

Table 3: The comparison between MCS vs. Pthreads/OpenMP
(highlighted rows indicate winning cases)

threads	num_iters	outside_work	n	mcs_ms	pthread_mutex_ms	omp_lock_ms
8	1000	1	53	1.7215	2.5995	42.4365
8	1000	1	89	2.0638	3.6733	45.1459
8	1000	1	101	2.8523	3.9499	45.2504
8	1000	1	503	8.4752	15.8361	48.6047
8	1000	3	53	1.5127	2.9158	43.084
8	1000	3	89	2.0434	3.9844	44.1593
8	1000	3	101	2.9108	4.3517	43.0496
8	1000	3	503	8.1613	14.5995	82.0425
8	1000	5	53	1.8106	3.6655	53.6617
8	1000	5	89	2.2278	4.3861	44.3493
8	1000	5	101	2.4722	4.705	47.2892
8	1000	5	503	9.3016	23.1942	53.8557
8	1000	7	53	1.8671	3.3134	43.794
8	1000	7	89	2.7945	4.1109	45.6914
8	1000	7	101	3.5367	4.5026	47.2965
8	1000	7	503	10.046	15.4174	51.7914
12	1000	1	53	3.6871	4.2437	65.5452
12	1000	1	89	5.0121	6.2619	71.636
12	1000	1	101	3.4828	6.2556	65.3198
12	1000	1	503	13.7857	23.4934	70.167
12	1000	3	53	7.9064	4.6005	64.3814
...						

For the MCS lock, I examined a regime with many threads, short critical sections, and varying contention. I fixed the number of iteration to 1000, used thread counts 8, 12, set outside work to 1, 3, 5, 7, and chose small primes such as 53, 89, 101, 503 for the critical-section work.

Table 3 (see full version under *lock_comparison* folder) describes that across all tested configurations MCS consistently achieved the lowest execution time, often substantially faster than both Pthreads and OpenMP. This behavior matches the theoretical advantages of MCS: threads form a queue and spin on their own local flags, which minimizes cache-coherence traffic and provides FIFO handoff. I, therefore, conclude that, in our multi-threaded, short-CS workload with moderate contention, the MCS lock clearly outperforms the standard Pthreads and OpenMP locking primitives on this machine.

2. Barrier Synchronization

2.1. Implement Sense-Reversing Barrier (SRB)

See the attachment (under *barrier-sync* folder)

2.2. Test the correctness

To test correctness, I designed a reusable multi-phase test that forces threads to synchronize repeatedly and checks whether any thread can pass the barrier before all others have arrived. The test runs with several thread counts and many barrier episodes so that reuse bugs show up reliably.

The workflow will look like this:

- For each phase, every thread increments a shared atomic counter *arrivals[phase]* and then calls *arrive_and_wait()*.
- After the barrier returns, each thread reads *arrivals[phase]* and verifies that its value equals the total number of threads. If the value is smaller, that means at least one thread was released too early.
- A second barrier call is then used before moving to the next phase, ensuring that all threads finish checking the current phase before any thread starts modifying the next one.
- The test records whether any phase fails and reports OK or FAIL after all threads complete.

This pattern is repeated for a large number of barrier episodes and for various thread counts (e.g., 2, 4, 8, 16, 20) to increase the chance of revealing race conditions.

The test code is presented in the attachment (*test_barrier_sync.cpp*)

2.3. Performance and Scalability Analysis

To compare the self-implemented SRB, the Pthreads barrier, and the OpenMP barrier, I use throughput, defined as the number of completed episodes per millisecond (episodes/ms). In all experiments each thread executes 10,000 episodes, and the execution time is measured from the moment the worker threads are started until all threads have finished all episodes. For each configuration (thread count and barrier type) the benchmark is repeated 20 times. The reported throughput is the mean of the 20 runs.

As shown in figure 4, when the number of threads does not exceed the number of hardware cores, SRB achieves much higher throughput than both the Pthreads and OpenMP barriers. The main reasons are that SRB is a very lightweight, busy-waiting barrier specialized for a fixed number of threads, while the library barriers implement more general semantics, use additional bookkeeping, all of which increase per-episode overhead in this tight micro-benchmark.

For all three barrier types, throughput decreases as the thread count grows from 2 to 24. Each episode must coordinate more arrivals, which increases cache-coherence traffic on shared state (counters, flags) and makes the overall episode latency more sensitive to the slowest thread. These effects reduce the number of episodes that can be completed per unit time.

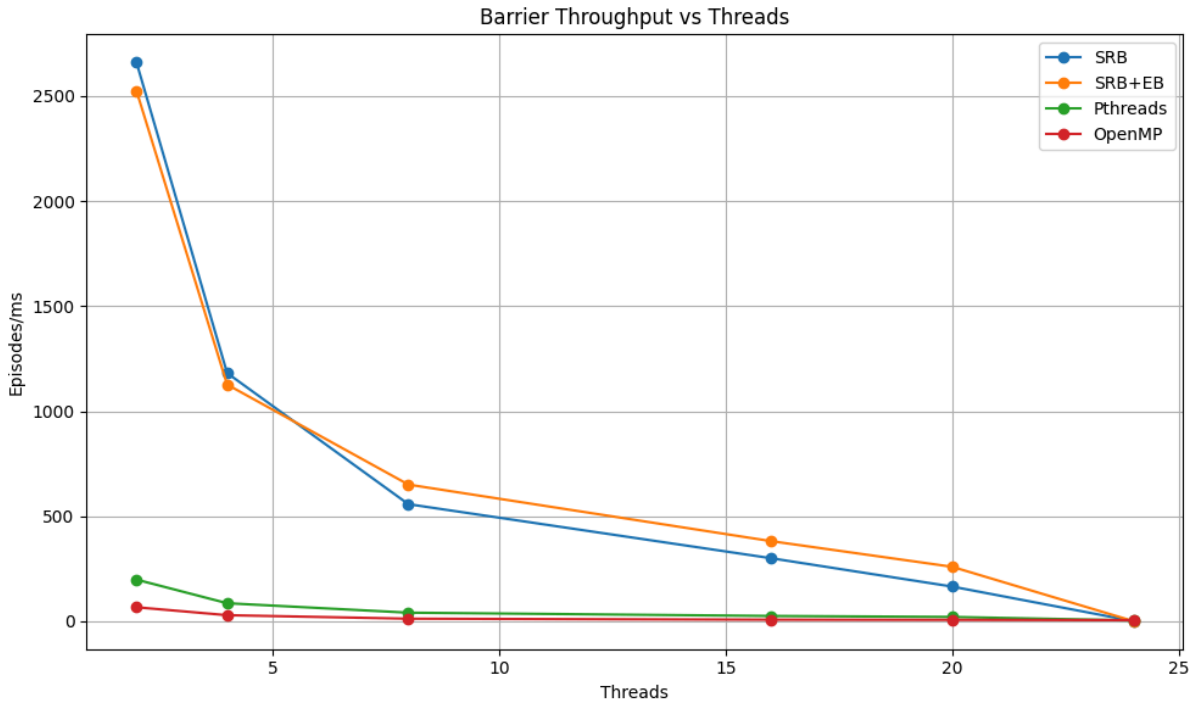


Figure 4: The comparison between different barrier throughput when the number of threads is increasing.

In oversubscribed configurations SRB performs worse than the Pthreads and OpenMP barriers (See figure 5). Because SRB is purely spin-based, many runnable threads busy-wait simultaneously, causing heavy contention for CPU time and cache resources (even with exponential backoff). In contrast, the library barriers employ spin-then-block strategies that allow the operating system to deschedule waiting threads, which reduces contention and leads to better throughput and system efficiency under oversubscription.

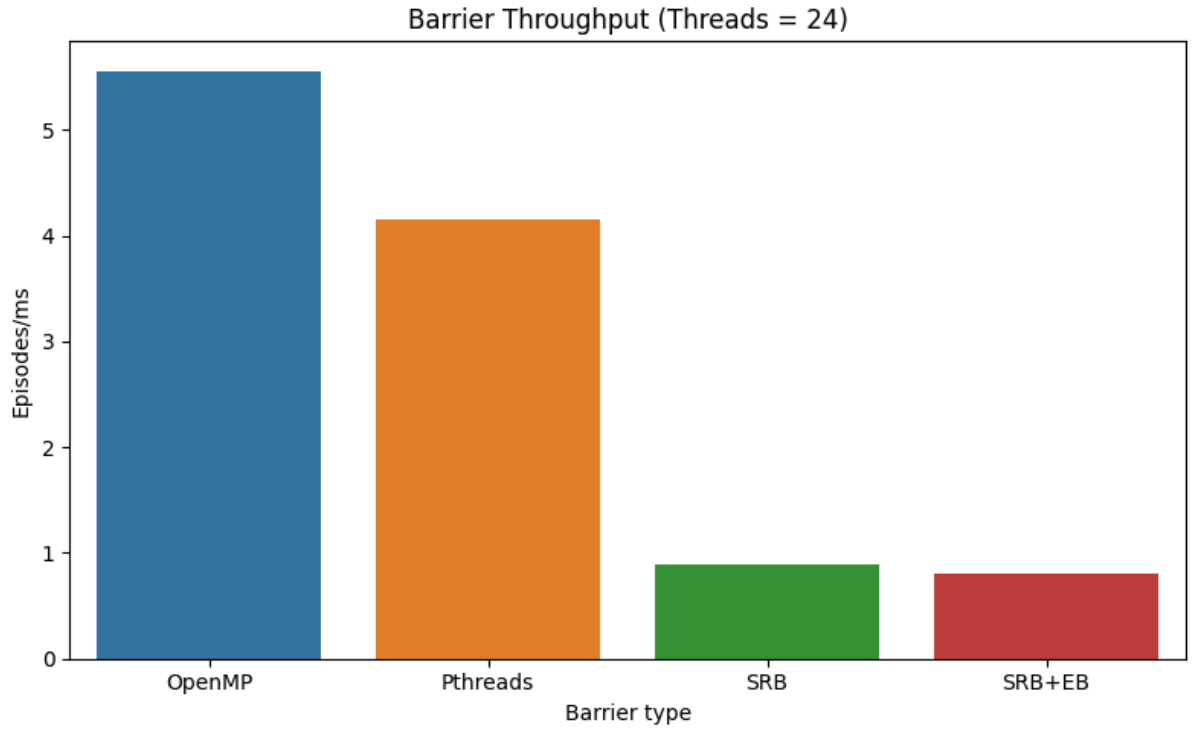


Figure 5: Barrier throughput under oversubscription (24 threads)

In summary, using episodes per millisecond as the metric, SRB clearly outperforms both the Pthreads and OpenMP barriers for thread counts that do not exceed the available hardware cores. However, this advantage does not translate into unlimited scalability. As I increase the number of threads, the cost of coordinating all arrivals, the cache-coherence traffic on the shared counter and sense flag, and the sensitivity to the slowest thread grow rapidly, so the throughput of SRB drops and eventually converges toward or can even fall below the more sophisticated ones like Pthreads and OpenMP barriers.

3. Concurrent Counters

3.1. Implement concurrent counters using mutex locks, compare-and-swap, fetch-and-add.

See the attachment (under *concurrent-counter* folder)

3.2. Test the correctness

To test correctness of the concurrent counter implementations, I verify that the final counter value equals the total number of increments performed by all threads under

various contention levels. This ensures that no increments are lost or duplicated, which is the key safety property of a concurrent counter.

The workflow is as follows:

- For each configuration, a fresh counter instance is created, then T worker threads are started, and each thread executes a loop of N calls to increment.
- After all threads finish and join, the program reads the final value and compares it against the expected value T × N, reporting a detailed error message if any mismatch occurs.

The test is executed with several settings to cover different contention scenarios: thread counts T ∈ {1, 2, 4, 8, 16, 20} and increments per thread N ∈ {10^4, 10^5, 10^6} for each of the three counter types (mutex-based, CAS-based, and fetch-and-add). All combinations pass without errors, indicating that all implementations behave correctly for the tested ranges of threads and operations.

The test code is presented in the attachment (*test_concurrent_counter.cpp*)

3.3. Performance and Scalability Analysis

The benchmark evaluates the execution time and scalability of three shared-counter implementations under multi-threaded increment workloads: mutex-based counter (MutexCounter), CAS-based counter (CASCounter), and atomic fetch-and-add counter (FetchAddCounter). For the mutex and CAS variants, two configurations are tested: original and exponential backoff enabled.

For each run, the program spawns T worker threads, where T ∈ {1,2,4,8,12,16,20}. Each thread performs 100,000 increment operations on the same shared counter instance. Hence, the total number of operations per run is T×100,000. Wall-clock runtime is measured in milliseconds, and is the mean of 20 repeated running times to improve statistical reliability.

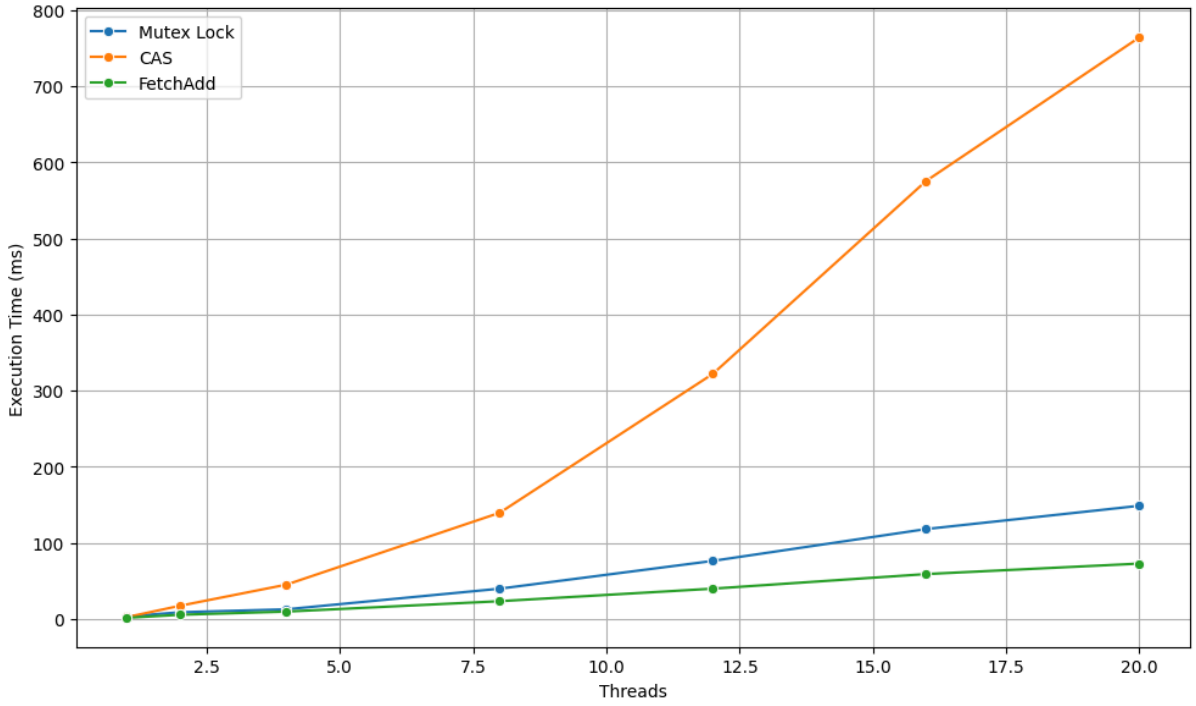


Figure 6: The execution time of three concurrent counter variants when the number of threads increases.

As the figure 6 shown, the fetch-add counter clearly delivers the best performance, followed by the mutex-based counter, while the CAS-based counter is the slowest for all tested thread counts. The fetch-add variant maintains the lowest execution time because it uses a single atomic read-modify-write operation that hardware can handle efficiently even under contention, so its runtime grows comparatively slowly as more threads are added.

The mutex-based implementation outperforms the CAS-based one because the mutex spends *less total work per successful increment* than the CAS loop, even though both ultimately serialize updates one after another. Under high contention, the CAS implementation has every thread repeatedly executing failed compare-and-swap operations on the shared counter until one succeeds, so it generates many more atomic operations and cache-coherence messages than the number of actual increments performed. By contrast, the mutex arranges contending threads in an internal queue and lets only one thread at a time enter the critical section to update the counter, while the others block or spin only briefly. This reduces the number of active contenders touching the hot cache line and therefore reduces wasted retries and coherence traffic. Because the mutex coordinates this serialized access with fewer failed attempts and less hardware contention, it ends up serializing more efficiently and performs better than the naive CAS-based counter.

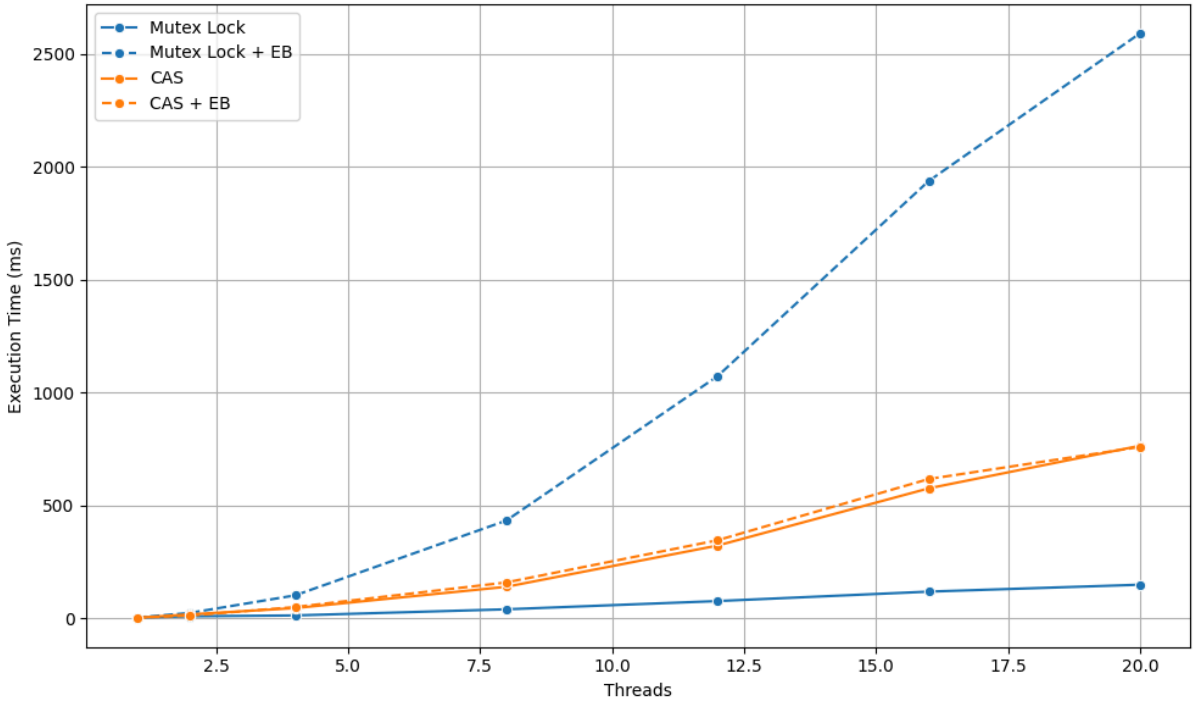


Figure 7: The execution time of mutex lock-based and CAS-based counters with and without exponential backoff for different thread counts

The exponential-backoff variants perform worse than the base mutex and CAS implementations in the benchmark (see figure 7). Each critical section consists of only a single increment, the lock or CAS is released again almost immediately, so the added sleeping or yielding after a failed attempt quickly dominates the useful work. Backoff methods are beneficial only when lock hold times are long enough that delaying retries significantly reduces contention. However, in this workload, the backoff primarily injects extra delay without sufficiently reducing contention on the shared counter.

In summary, the experiment shows that the atomic fetch-and-add counter consistently achieves the best performance and scalability, the mutex-based counter is slower but still reasonable, and the CAS-based counter performs the worst at higher thread counts. The exponential-backoff variants of both mutex and CAS further degrade performance for this specific workload, because the critical section is extremely short and the added delays after failed attempts only stretch out the same serialized sequence of increments.

4. References

- Anderson, T. E. (1990). The performance of spin lock alternatives for shared-money multiprocessors. *IEEE Transactions on Parallel and Distributed Systems*, *1*(1), 6–16. https://doi.org/10.1109/71.80120
- Karl Fürlinger, *Condition Synchronization in Parallel Computing* (Advanced Topics in Parallel Computing, LMU Munich, 2024)
- Karl Fürlinger, *Memory Consistency Models* (Advanced Topics in Parallel Computing, LMU Munich, 2024)
- Karl Fürlinger, *Spinlocks for Mutual Exclusion* (Advanced Topics in Parallel Computing, LMU Munich, 2024)
- Karl Fürlinger, *The C Memory Model and Atomic Operations, Parts 1–2* (Advanced Topics in Parallel Computing, LMU Munich, 2024)